

TECHNICAL DOCUMENT

Delivery Navigation ROS 2 System

1. System Overview

The Delivery Navigation system is a ROS 2 based simulation of a delivery robot moving toward a destination.

The system demonstrates:

- ROS 2 topics
- ROS 2 services
- Custom messages
- Custom service definitions
- Publisher and subscriber communication
- Service client and service server communication
- Launch files
- Object-oriented programming
- Separation of responsibilities between interfaces, services, and nodes

The implementation uses **ROS 2 Humble**, **Python**, and an **OOP architecture**.

2. System Architecture

The system contains two ROS 2 packages:

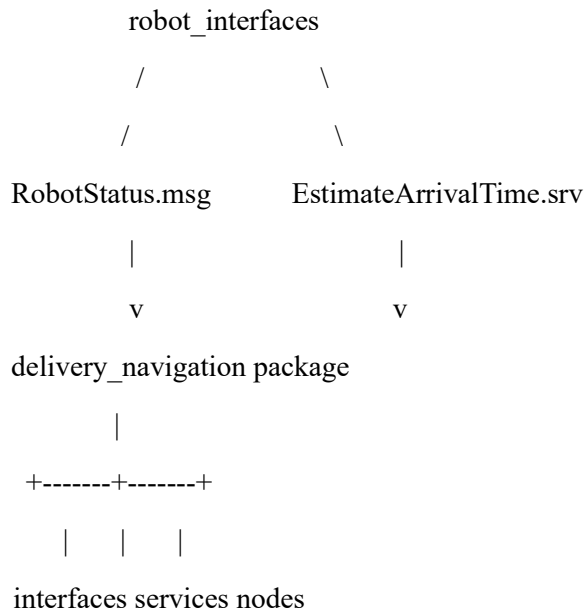
ros2_workspace/

└─ src/

 └─ delivery_navigation/

 └─ robot_interfaces/

The relationship between the packages is:



The `robot_interfaces` package contains the ROS 2 custom communication definitions.

The `delivery_navigation package` contains the application logic and ROS 2 nodes.

3. Custom Message

The system uses a custom message called:

`RobotStatus.msg`

The message contains:

`float32 distance_remaining`
`float32 speed`
`bool obstacle_detected`

Fields

distance_remaining

The remaining distance between the robot and the destination in meters.

speed

The current robot speed in meters per second.

obstacle_detected

Indicates whether an obstacle is currently detected.

This message is published on the:

`/robot_status`

topic.

4. Custom Service

The system uses the following service:

EstimateArrivalTime.srv

Definition:

float32 distance_remaining

float32 speed

float32 minutes_remaining

The request contains:

- Remaining distance
- Robot speed

The response contains:

- Estimated time remaining in minutes

The service is available through:

/estimate_arrival_time

5. Package Structure

The main application package is organized as follows: delivery_navigation/

```
|— delivery_navigation/
|  |— interfaces/
|  |  |— navigation_interface.py
|  |  |— status_interface.py
|  |  |— arrival_estimator.py
|  |  |— arrival_client_interface.py
|  |
|  |— services/
|  |  |— navigation_service.py
|  |  |— status_service.py
|  |  |— arrival_time_service.py
```

```
| | └─ arrival_client_service.py
| |
| └─ nodes/
|   └─ nav_publisher.py
|   └─ nav_subscriber.py
|   └─ nav_service_server.py
|
└─ launch/
  └─ navigation_launch.py
|
└─ package.xml
└─ setup.py
```

6. Object-Oriented Design

The project uses abstract interfaces to define the responsibilities of the application services.

6.1 NavigationInterface

NavigationInterface defines the method:

```
get_next_status()
```

Its implementation is provided by:

NavigationService

The service is responsible for generating the next robot status.

6.2 StatusInterface

StatusInterface defines:

```
process_status(msg)
```

Its implementation is:

StatusService

This service converts the received robot status into an appropriate message for the user.

For example, it generates a warning when an obstacle is detected.

6.3 ArrivalEstimator

ArrivalEstimator defines:

`estimate(distance, speed)`

Its implementation is:

`ArrivalTimeService`

The service calculates:

$\text{time} = \text{distance} / \text{speed} / 60$

If the speed is zero or negative, the service returns:

0.0

to avoid an invalid division.

6.4 ArrivalClientInterface

`ArrivalClientInterface` defines:

`create_request(distance, speed)`

Its implementation is:

`ArrivalClientService`

It prepares the data used by the ROS 2 service client before sending the request.

7. Navigation Publisher

The `NavPublisher` node is responsible for publishing robot status messages.

It creates a publisher for:

`/robot_status`

with:

`RobotStatus`

as the message type.

The node also uses a timer with a period of:

1 second

At every timer event, the navigation service generates a new status message and publishes it.

The initial navigation values are:

$\text{distance} = 200.0 \text{ m}$

$\text{speed} = 2.0 \text{ m/s}$

After publishing each status, the remaining distance is updated:

$\text{distance} = \max(0.0, \text{distance} - \text{speed})$

Therefore, the simulated robot moves **2 meters every second**.

With an initial distance of 200 meters and a speed of 2 m/s, the initial estimated arrival time is:

$200 / 2 = 100$ seconds

$100 / 60 = 1.67$ minutes

8. Navigation Subscriber

The NavSubscriber node subscribes to:

/robot_status

When a new message is received, the subscriber performs two operations.

Step 1: Process the robot status

The subscriber passes the message to:

StatusService

The service generates either:

Distance: ... | Speed: ...

or:

WARNING: Obstacle detected! Distance: ...

Step 2: Request estimated arrival time

The subscriber creates a service request using:

ArrivalClientService

The request contains:

distance_remaining

speed

The request is sent asynchronously to:

/estimate_arrival_time

When the response is received, the subscriber displays:

Estimated arrival time: ... minutes

9. Arrival Time Service Server

The NavServiceServer node provides the:

/estimate_arrival_time

service.

The server receives:

distance_remaining

speed

and passes them to:

ArrivalTimeService

The service calculates:

$\text{distance} / \text{speed} / 60$

and returns the result as:

minutes_remaining

The server therefore separates ROS 2 communication from the actual business logic.

10. Communication Flow

The complete communication flow is:

NavPublisher

|

| RobotStatus

v

/robot_status

|

v

NavSubscriber

|

| process status

v

StatusService

NavSubscriber

|

| EstimateArrivalTime request

v

`/estimate_arrival_time`

|

v

NavServiceServer

|

v

ArrivalTimeService

|

| `minutes_remaining`

v

NavSubscriber

The subscriber therefore acts as both:

- A subscriber to the robot status topic
- A client of the arrival time service

11. Launch System

The project includes: `navigation_launch.py`

The launch file starts all three main nodes:

`nav_publisher`

`nav_service_server`

`nav_subscriber`

This allows the complete application to be started using one command:

`ros2 launch delivery_navigation navigation_launch.py`

12. Error Handling

The nodes use:

try:

`rclpy.spin(node)`

except `KeyboardInterrupt`:

`pass`

finally:

`node.destroy_node()`

The `rcipy.shutdown()` call is not performed inside the individual nodes because the nodes may be launched and managed by a ROS 2 launch system that handles the shutdown of the ROS 2 context.

This prevents duplicate shutdown attempts when the user stops the launch system using Ctrl + C.

The subscriber also checks whether the arrival time service is available before sending a request.

13. Build and Execution Commands

Build the interface package:

```
colcon build --packages-select robot_interfaces
```

Build the application package:

```
colcon build --packages-select delivery_navigation
```

Source the workspace:

```
source install/setup.bash
```

Run the complete system:

```
ros2 launch delivery_navigation navigation_launch.py
```

14. Testing

The custom message can be inspected using:

```
ros2 interface show robot_interfaces/msg/RobotStatus
```

The custom service can be inspected using:

```
ros2 interface show robot_interfaces/srv/EstimateArrivalTime
```

The arrival time service can be tested manually:

```
ros2 service call /estimate_arrival_time robot_interfaces/srv/EstimateArrivalTime  
"{distance_remaining: 20.0, speed: 2.0}"
```

Expected result:

```
minutes_remaining: 0.16666667
```

This corresponds to:

$$20 / 2 / 60 = 0.1667 \text{ minutes}$$

This verifies the service calculation independently from the publisher and subscriber nodes.

15. Design Benefits

The architecture separates the system into three main layers:

Interfaces



Services



ROS 2 Nodes

This separation provides:

- Clear responsibility for each component
- Easier testing
- Reusable business logic
- Better maintainability
- Reduced complexity inside ROS 2 nodes
- Compliance with the required OOP structure

The ROS 2 nodes mainly handle communication, while the application services handle the actual processing and calculations.

16. Conclusion

The Delivery Navigation system demonstrates a complete ROS 2 communication workflow using a custom topic, a custom service, a service client, and a launch file.

The project combines ROS 2 communication with object-oriented design by separating interfaces, service logic, and ROS 2 nodes. This structure makes the system easier to understand, test, and extend for future navigation features.